

Experiment 4 : Developing REST APIs using Amazon API Gateway

What is Amazon API Gateway ?

Think of API Gateway as a **main door (gate)** of your application.

- Users (apps, browser, mobile) → come to this door
- API Gateway → sends request to backend (like AWS Lambda)
- Then sends response back to user

API Gateway = **middleman between user and backend**

Key Features

1. API Creation

You can create APIs easily

Example:

- /login
- /hello

2. Integration with AWS

It connects with:

- Lambda (most common)
- Database (like Amazon DynamoDB)

3. Security

You can control who can use your API:

- API Key
- Login (Auth)
- IAM roles

4. Traffic Control

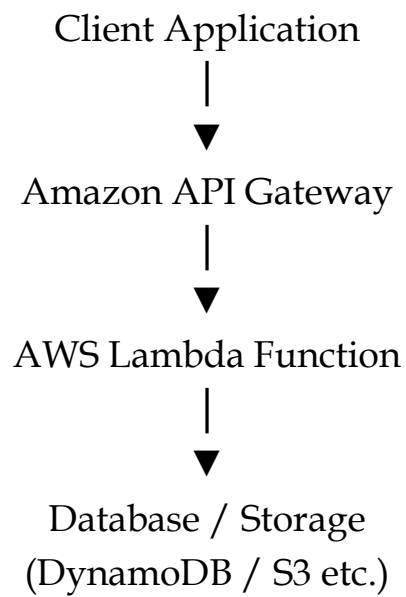
Prevent overload:

- Limit requests (throttling)
- Cache responses

5. Monitoring

You can track API activity using
Amazon CloudWatch

Architecture of REST API with API Gateway



Components

1. Client

User or app

Example:

- Browser
- Mobile app

2. API Gateway

Receives request

Decides where to send it

Example:

GET /hello

3. Backend (Lambda)

Processes request

Runs code

Example:

return "Hello World"

4. Database

Stores data

Examples:

- DynamoDB
- S3

Steps to Develop REST API using API Gateway

Step 1: Create a Lambda Function

- Login to AWS Console.
- Open AWS Lambda.
- Click Create Function.

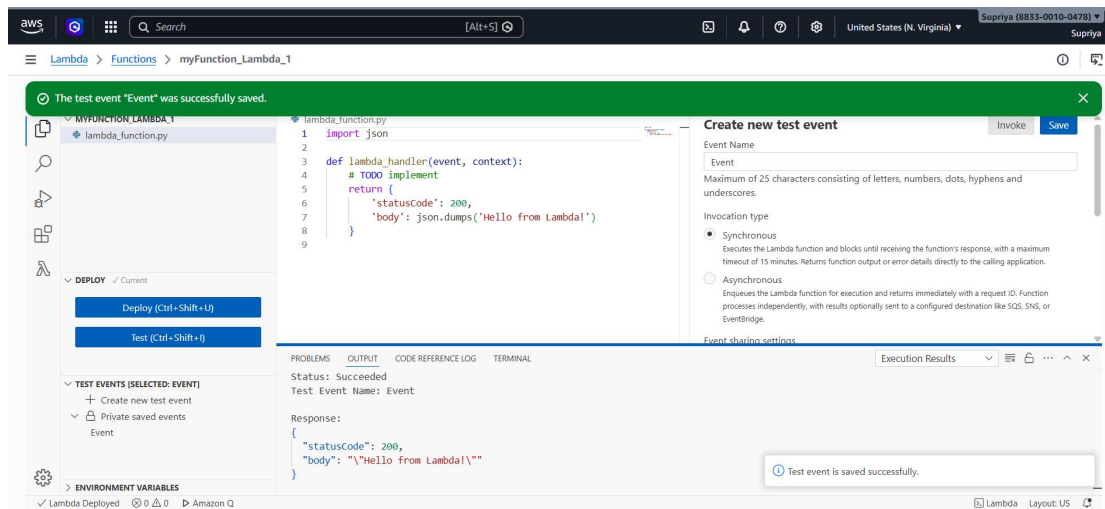
The screenshot shows the AWS Lambda 'Create function' page. At the top, there are three tabs: 'Author from scratch' (selected), 'Use a blueprint', and 'Container image'. Below these, the 'Basic information' section contains a 'Function name' field with the value 'myFunction_Lambda_1'. The 'Runtime' dropdown is set to 'Python 3.14'. The 'Architecture' dropdown is set to 'x86_64'. The 'Durable execution' checkbox is unchecked. The 'Permissions' section shows the default role.

- Choose **Author from scratch**.

Write a simple function.

Example (Python):

```
import json
def lambda_handler(event, context):
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
```



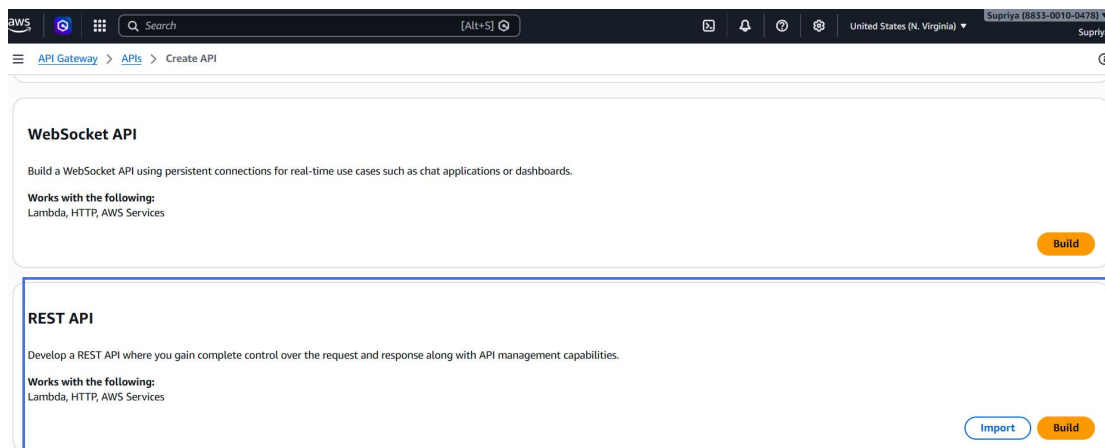
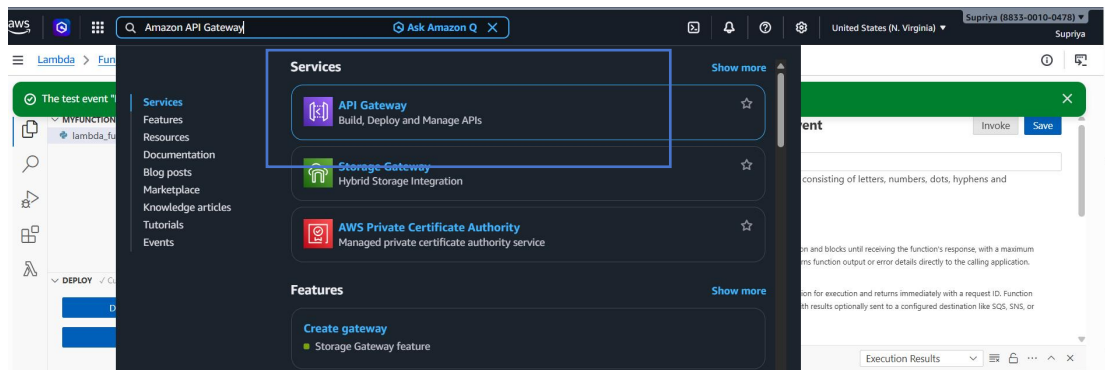
Step 2: Create REST API

Open Amazon API Gateway.

Click **Create API**.

Select **REST API**.

Click **Build**.



Create REST API Info

API details

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name
StudentAPI

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.
Regional

Security policy - new Info
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.

Step 3: Create Resource and Method

Click **Actions** → **Create Resource**.

Resource Name: */hello*.

Click **Create Method** → **GET**.

Create resource

Successfully created REST API 'StudentAPI (4x0hlf5aj2)'.

Resource details

☒ **Proxy resource** Info
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path
/

Resource name
hello

☐ **CORS (Cross Origin Resource Sharing)** Info
Create an OPTIONS method that allows all origins, all methods, and several common headers.

Cancel Create resource

Successfully created resource '/hello'

Create method

Method details

Method type
GET

Integration type

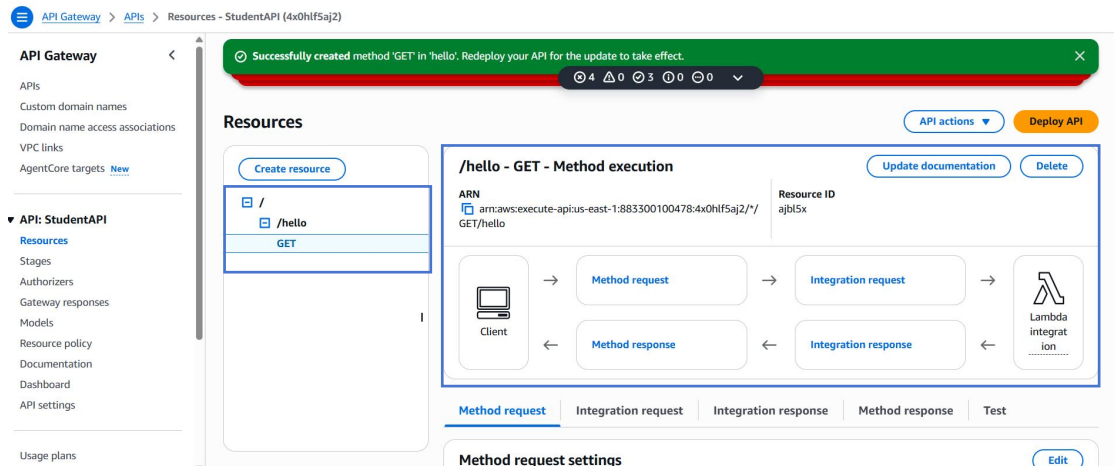
☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **Mock**
Generate a response based on API Gateway m.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.



Step 4: Integrate with Lambda

Integration Type: **Lambda Function**.

Select your Lambda function.

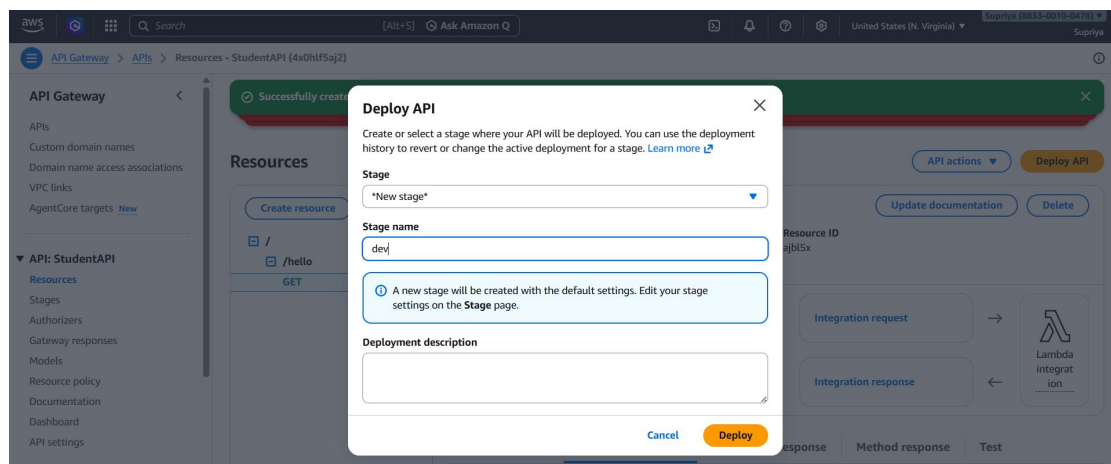
Save and allow permissions.

Step 5: Deploy the API

Click **Actions** → **Deploy API**.

Create a **Stage** (example: *dev*).

Deploy.



You will get an **Invoke URL** like:

<https://daz0is6dbj.execute-api.us-east-1.amazonaws.com/dev/hello>

